

Expose `std::basic-format-string<charT, Args...>`

Document #: P2508R2
Date: 2022-02-04
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[2.1 Why exposition-only?](#)

[2.2 Yet another DR against C++20, are you serious?](#)

[2.3 Also need a getter](#)

[2.4 Feature-test macro?](#)

[3 Wording](#)

[3.1 Feature-test macro](#)

[4 References](#)

§ 1 Revision History

Since [\[P2508R1\]](#): wording update.

Since [\[P2508R0\]](#): making *basic-format-string* non-exposition only isn't sufficient, also actually need a way to pull out the underlying format string if we want to eventually invoke `vformat`.

§ 2 Introduction

[\[P2216R3\]](#), in order to improve type safety around the `std::format` API, introduced the type `std::basic-format-string<charT, Args...>` as a way to check to make sure that the format string actually matches the arguments provided. The example used in that paper is:

```
std::string s = std::format("{:d}", "I am not a number");
```

Originally, this would throw an exception at runtime. With the change, this now is a compile error. Which is a very nice improvement - catching bugs at compile time is awesome.

Unfortunately, the ability to validate format strings at compile time is currently limited to specifically those functions provided by the standard library. In particular, `std::format`, `std::format_to`,

`std::format_to_n`, and `std::formatted_size`. But users cannot take advantage of this in their own code.

As Joseph Thompson pointed out in [\[std-discussion\]](#):

However, consider the common use case:

```
template <typename... Args>
void log(std::format_string<Args...> s, Args&&... args) {
    if (logging_enabled) {
        log_raw(std::format(s, std::forward<Args>(args)...));
    }
}
```

Without a specified `std::format_string` type, is it possible to forward arguments to `std::format` with compile-time format string checks? I'm thinking about this as an average user of the standard library, so I'm not looking for workarounds. Is this a defect?

Today, if users want to statically verify their format strings... what can they do? Originally, I thought that you could write a macro to wrap the call to `log`, so that you can attempt to pass the expressions as they are to try to statically verify them. But this compiles just fine:

```
using T = decltype(std::format("{:d}", "I am not a number")); // ok
static_assert(requires { std::format("{:d}", "I am not a number"); }); // ok
```

Because... of course it compiles. The way that this check works in `format` today is that *basic-format-string*<charT, Args...> has a `constexpr` constructor that just isn't a constant-expression if the value of the format string doesn't match the arguments.

But constant-expression-ness (or, in general, values) don't affect overload resolution, so there's no way to check this externally (there were some papers that considered value-based constraints which would've let this work [\[P1733R0\]](#) [\[P2049R0\]](#), which aren't without potential issue [\[P2089R0\]](#), but we don't have something like that today).

The only way for Joseph (or you or me or ...) to get this behavior in `log` is to basically manually re-implement *basic-format-string*. That seems like an unreasonable burden to place for such a useful facility. The standard library already has to provide it, why not provide it under a guaranteed name that we can use?

§ 2.1 Why exposition-only?

There are two reasons why *basic-format-string*<charT, Args...> (and its more specialized alias templates, *format-string*<Args...> and *wformat-string*<Args...>) are exposition-only.

The first is that, as a late DR against C++20, [\[P2216R3\]](#) was trying to limit its scope. Totally understandable.

The second is that it is possible with future language features to be able to do this better. For example, if we had `constexpr` function parameters [\[P1045R1\]](#) or some kind of hygienic macro facility [\[P1221R1\]](#),

then the whole shape of this API would be different. For `constexpr` function parameters, for instance, the first argument to `std::format` would probably be a `constexpr std::string` (or some such) and then there would just be a `consteval` function you could call like `validate_format_string<Args...>(str)`. In which case, we could expose `validate_format_string` and the implementation of `log` could call that directly too. And that would be pretty nice!

But we won't have `constexpr` function parameters or a hygienic macros in C++23, so the solution that we have to this problem today is `basic-format-string<charT, Args...>`. Since we have this solution, and it works, we shouldn't just restrict it to be used by the standard library internally. If eventually we adopt a better way of doing this checking, we can expose that too. Better give users a useful facility today that's possibly worse than a facility we might be able to provide in C++26, rather than having them have to manually write their own version (which is certainly doable, though tedious and error-prone).

§ 2.2 Yet another DR against C++20, are you serious?

This doesn't strictly have to be a DR, and could certainly just be a C++23 feature. Although it would be nice to have sooner rather than later.

Even if an implementation already ships with a `std::_Ugly::_Basic_Format__Construct__Additional__Underscores<charT, Args...>` (which Microsoft is [close to doing](#), with a slightly more reasonable ugly name), they can still provide the new names (`std::basic_format_string`, `std::format_string`, and `std::wformat_string`) without having to worry about ABI, since these are templates. Or we could ask Charlie very nicely to simply rename `_Basic_format_string` to `basic_format_string` as that PR is going out the door.

Either way, it would be nice to avoid specifying `std::basic_format_string` as an alias to `std::basic-format-string` (which, technically, is an option, and obviously has no ABI impact or take much implementation effort at all). That would be weird, but probably not even particularly notably weird. Note that specifying `std::basic_format_string` as a class template does prohibit implementing it as an alias template to `std::_Basic_format_string`, as this difference is observable.

I leave it up to the discretion of LEWG, Charlie, and the ghost of C++20 Past whether or not we ever actually want to declare C++20 complete.

§ 2.3 Also need a getter

Making `basic-format-string` publicly accessible is enough to make the original example work, since `std::format` just takes a `std::format_string<Args...>`, so if we construct one in advance and pass it in, that's sufficient:

```
template <typename... Args>
void log(std::format_string<Args...> s, Args&&... args) {
    if (logging_enabled) {
        log_raw(std::format(s, std::forward<Args>(args)...));
    }
}
```

However, it's not enough for us with other APIs. We may want to do static validation in the front-end of our logger, but then ship everything over to the back-end to do actual logging, in a way that could support shipping the actual format string but not its type. This is still useful, as it lets us get the benefit of doing checks up front. It's just that ultimately, instead of calling `std::format` to do the actual formatting, we call `std::vformat` (which just takes a `string_view` and a `format_args`).

In order to do so, we need some way to pull out the underlying `string_view` from a `std::format<Args...>`. A simple getter suffices, but we do need that getter. So this paper isn't simply making names non-exposition-only, it's also adding a function.

§ 2.4 Feature-test macro?

It is definitely important to provide a feature-test macro for this, which will allow log above to conditionally opt in to this feature if possible:

```
#if __cpp_lib_format >= whatever
    template <typename... Args>
    using my_format_string =
        std::format_string<std::type_identity_t<Args>...>;
#else
    template <typename... Args>
    using my_format_string = std::string_view;
#endif

template <typename... Args>
void log(my_format_string<Args...> s, Args&&... args);
```

§ 3 Wording

In 20.20.1 [\[format.syn\]](#), replace the exposition-only names *basic-format-string*, *format-string*, and *wformat-string* with the non-exposition-only names `basic_format_string`, `format_string`, and `wformat_string`.

Do the same for their uses in 20.20.4 [\[format.fmt.string\]](#) (renaming the clause to “Class template `basic_format_string`”) and 20.20.5 [\[format.functions\]](#).

In 20.20.4 [\[format.fmt.string\]](#), the member should still be exposition only. The full subclause should now read (the only changes are the name of the class template, which no longer is exposition only, and the new getter):

```
template<class charT, class... Args>
struct basic_format_string {
private:
    basic_string_view<charT> str;           // exposition only

public:
    template<class T> consteval basic_format_string(const T& s);

    + constexpr basic_string_view<charT> get() const noexcept { return str; }
};
```

```
template<class T> constexpr basic_format_string(const T& s);
```

- 1 *Constraints:* `const T&` models `convertible_to<basic_string_view<charT>>`
- 2 *Effects:* Direct-non-list-initializes *str* with *s*.
- 3 *Remarks:* A call to this function is not a core constant expression ([`expr.const`]) unless there exist args of type `Args` such that *str* is a format string for *args*.

§ 3.1 Feature-test macro

Bump the format feature-test macro in 17.3.2 [[version.syn](#)]:

```
- #define __cpp_lib_format 202110L // also in <format>
+ #define __cpp_lib_format 2022XXL // also in <format>
```

§ 4 References

[P1045R1] David Stone. 2019-09-27. `constexpr` Function Parameters.

<https://wg21.link/p1045r1>

[P1221R1] Jason Rice. 2018-10-03. Parametric Expressions.

<https://wg21.link/p1221r1>

[P1733R0] David Sankel, Daveed Vandevoorde. 2019-06-17. User-friendly and Evolution-friendly Reflection: A Compromise.

<https://wg21.link/p1733r0>

[P2049R0] Andrew Sutton, Wyatt Childers. 2020-01-13. Constraint refinement for special-cased functions.

<https://wg21.link/p2049r0>

[P2089R0] Barry Revzin. 2020-02-17. Function parameter constraints are too fragile.

<https://wg21.link/p2089r0>

[P2216R3] Victor Zverovich. 2021-02-15. `std::format` improvements.

<https://wg21.link/p2216r3>

[P2508R0] Barry Revzin. 2021-12-17. Exposing `std::basic_format_string`.

<https://wg21.link/p2508r0>

[P2508R1] Barry Revzin. 2022-01-18. Exposing `std::basic_format_string`.

<https://wg21.link/p2508r1>

[`std-discussion`] Joseph Thomson. 2021. Should a `std::basic_format_string` be specified?

<https://lists.isocpp.org/std-discussion/2021/12/1526.php>